Agente de Gestión de Casos CESFAM

Agente funcional con LangGraph: herramientas de consulta, razonamiento y escritura, memoria y planificación adaptativa

Autor	Docente	Fecha	Repositorio
Oscar Díaz Ulloa	Jaime Menéndez	02-11-2026	github.com/osdiazu/ISY0101-ep02-oscar-diaz

1. Requerimiento organizacional

En la EP1 se construyó un asistente documental RAG para el CESFAM, que responde con cita verificable y se abstiene cuando no tiene respaldo. La EP2 pasa de **responder preguntas** a **gestionar casos completos**. La OIRS y el SOME reciben reclamos, solicitudes de copia de ficha, consultas y alertas de inasistencia. Cada caso exige consultar protocolos, calcular plazos hábiles, priorizar, registrar, derivar y responder, y puede durar varios días. El agente automatiza ese flujo de punta a punta: deja un **borrador para firma de la jefatura** y todas las tareas y documentos registrados en el sistema OIRS (simulado en SQLite).

2. Diseño e implementación del agente (IE1, IE2)

El agente es un **grafo de estados de LangGraph** (`StateGraph`) con seis nodos y una arista condicional (Figura 1). El estado `EstadoCaso` funciona como memoria de trabajo del caso: guarda el plan, los hechos calculados, los eventos y la bitácora, y se persiste después de cada paso. Las herramientas se definen con `@tool` de `langchain_core`: tienen firma tipada, descripción y salida JSON. Se agrupan en tres familias para separar las que tienen efectos laterales (Tabla 1). El conocimiento institucional proviene del RAG híbrido de la EP1 (BM25 + denso + *Reciprocal Rank Fusion*), expuesto como la herramienta `buscar_normativa`.

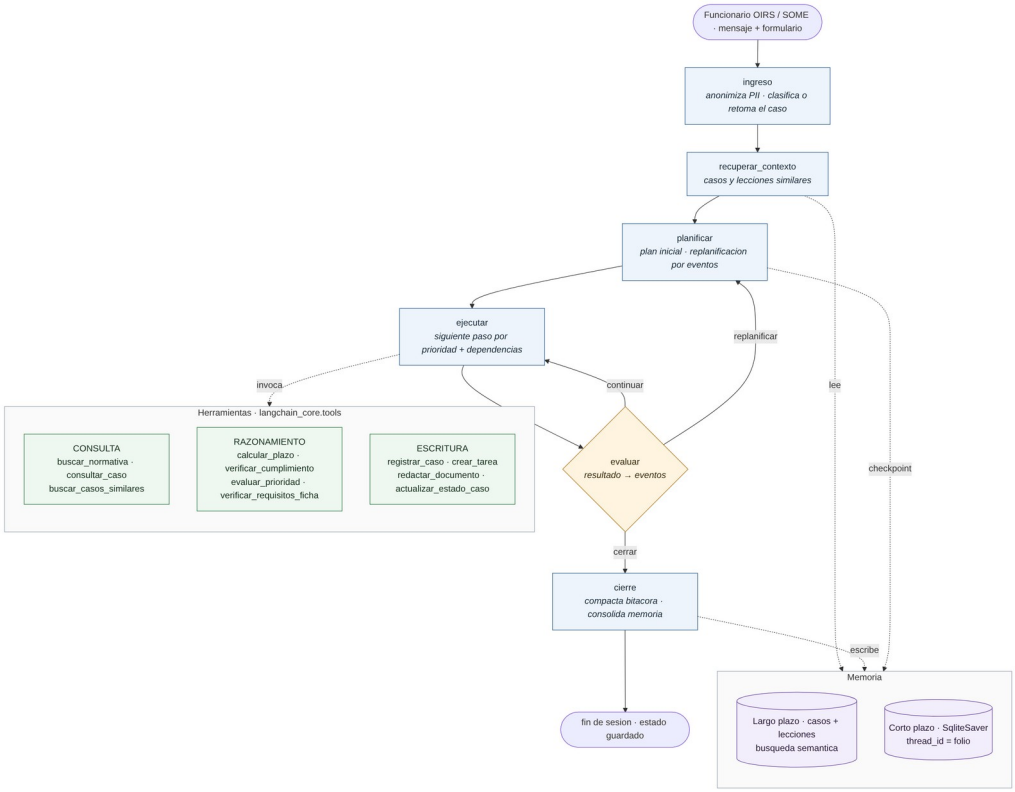


Figura 1. Orquestación de componentes: nodos del grafo, herramientas y memorias.

Familia	Herramientas	Función en el flujo
Consulta	`buscar_normativa`, `consultar_caso`, `buscar_casos_similares`	Protocolos y normas con cita; estado del folio; casos parecidos en memoria
Razonamiento	`calcular_plazo`, `verificar_cumplimiento`,	Días hábiles con feriados, plazos vencidos, prioridad con

Familia	Herramientas	Función en el flujo
	`evaluar_prioridad`, `verificar_requisitos_ficha`	puntaje explicable, requisitos de PRO-006
Escritura	`registrar_caso`, `crear_tarea`, `redactar_documento`, `actualizar_estado_caso`	Registro OIRS, derivaciones con fecha límite, respuesta con estructura MAN-003, cambios de estado

Tabla 1. Las 11 herramientas del agente.

Frameworks. LangGraph ≥ 0.6 (orquestación, *reducers*, *checkpointing*), `langgraph-checkpoint-sqlite` y `langchain-core`. **Escalabilidad:** `SqliteSaver` se reemplaza por `PostgresSaver`, y el almacén de memoria por PostgreSQL con pgvector, sin modificar los nodos. **Compatibilidad:** las *tools* son reutilizables en cualquier agente LangChain. Un flujo de GitHub Actions instala las dependencias y ejecuta pruebas y escenarios en Python 3.10 y 3.12. El sistema funciona sin GPU y sin clave de API; con `ANTHROPIC_API_KEY` la redacción usa un LLM real.

3. Memoria y recuperación de contexto (IE3, IE4)

- **Corto plazo (continuidad del caso).** El *checkpointer* `SqliteSaver` guarda el estado completo con `thread_id = folio`. Un caso que queda *pendiente de antecedentes* se retoma días después, en otra instancia del proceso, con el plan, los pasos suspendidos y los plazos intactos (Figura 2). La bitácora usa un *reducer* propio: agrega entradas y, sobre 40, las compacta en un resumen que conserva los hitos y las 10 más recientes. Así se controla la ventana de contexto en flujos prolongados.
- **Largo plazo (aprendizaje entre casos).** Al cerrar un caso se guardan su resumen **anonimizado** y las lecciones de sus eventos relevantes en un almacén vectorial con espacios `casos` y `lecciones`. Al abrir un caso nuevo se recuperan los similares por coseno. En el escenario E6, un reclamo que no menciona GES recupera la lección del E1 (similitud 0,56) y el planificador agrega una verificación preventiva del FCIP.
- **Recuperación semántica de conocimiento.** Es el RAG de la EP1, con gate de confianza y verificación de citas. Se corrigió el corte de oraciones del motor extractivo y la cobertura del dato clave subió de **0,737 a 0,947**, con recall@4 = 1,0 y 0 fugas de datos personales.

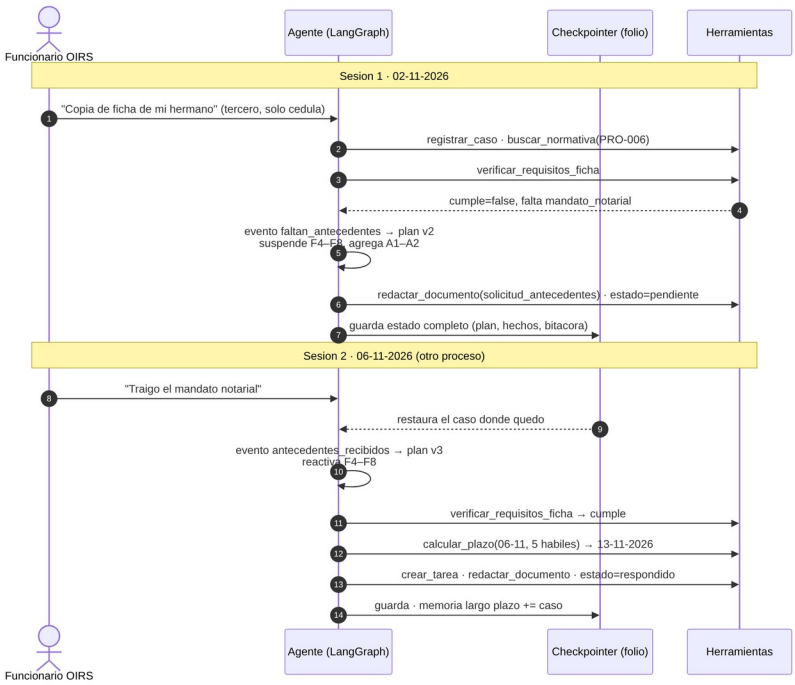


Figura 2. Caso E2 en dos sesiones: el checkpoint permite suspender y retomar el flujo.

4. Planificación y toma de decisiones (IE5)

Cada paso del plan declara `accion`, `args`, `prioridad` (0 crítica a 3 baja), `depende_de`, `proposito` y `origen` (plantilla, memoria o `replan:<evento>`). El siguiente paso es el **pendiente de menor prioridad con dependencias resueltas**: una escalación crítica insertada a mitad del flujo se ejecuta antes que la redacción de la respuesta. Después de cada herramienta, el nodo `evaluar` aplica reglas que convierten resultados en **eventos**, y `planificar` transforma el plan en respuesta a ellos (Tabla 2).

Esc.	Condición que cambia	Decisión del agente	Resultado
E1	FCIP emitido fuera de 24 h hábiles (8 días de atraso)	Tarea crítica a Subdirección Clínica; respuesta sube a prioridad 1	respondido · crítica
E2	Tercero sin mandato notarial → lo trae 4 días después	Suspende la rama de entrega; luego la reactiva y cuenta el plazo desde la solicitud completa	pendiente → respondido (13-11)
E3	RAG cita MAN-003 §5 en reclamo por trato	Inserta tarea y minuta a Dirección el mismo día (prioridad 0)	escalado · crítica
E4	RAG se abstiene: sin respaldo documental	Omite la respuesta; deriva y emite acuse	derivado
E5	Llamada sin contacto; luego feriado nuevo	Visita domiciliaria a 15 días hábiles; recalcula plazo 30-11 → 01-12	3 sesiones, plan v3
E6	Lección similar en memoria	Agrega verificación GES preventiva (`origen = memoria`)	plan modificado
E7	Falla del sistema OIRS	Reintenta (máx. 2) y, si persiste, escala a Jefatura OIRS	respondido

Tabla 2. Escenarios de prueba y comportamiento adaptativo observado (evidencias/demo_escenarios.txt).

Pruebas. 49 de 49 aprobadas: 21 del RAG de la EP1 y 28 del agente. Cubren herramientas, selección y replanificación, memoria (incluida la reanudación en un proceso nuevo), ausencia de datos personales en documentos y memoria, y los siete escenarios de extremo a extremo. El tope de 30 iteraciones por sesión y el `recursion_limit` del grafo evitan bucles.

5. Justificación de las decisiones de diseño

Se eligió LangGraph porque permite controlar explícitamente los estados, transiciones y persistencia del agente, siendo más adecuado para flujos administrativos que requieren trazabilidad que alternativas como ReAct o AutoGen. El planificador basado en reglas evita delegar completamente las decisiones al LLM, reduciendo errores y asegurando que las prioridades y dependencias se respeten, mientras que la replanificación por eventos permite adaptarse a situaciones imprevistas.

La memoria se dividió en corto y largo plazo para mantener la continuidad de los casos y reutilizar experiencias anteriores. SQLite fue seleccionado por su simplicidad y facilidad de implementación, permitiendo migrar posteriormente a PostgreSQL. Finalmente, los plazos se calculan mediante código determinista y no con el LLM, garantizando mayor precisión en días hábiles y feriados.

Como limitación, las reglas actuales cubren únicamente los casos definidos y requieren ajustes ante nuevos procedimientos o cambios en la interpretación de plazos y feriados.

6. Conclusiones y reflexión individual

La EP2 logró evolucionar el RAG de la EP1 hacia un agente capaz de gestionar casos, consultar normativa, calcular plazos, registrar tareas y generar respuestas con supervisión humana. Las 49 pruebas aprobadas y los siete escenarios demostraron el funcionamiento de la solución.

Como limitaciones, la integración OIRS es simulada y la cobertura depende de las reglas implementadas. Como siguiente etapa, se propone integrar un sistema real, ampliar los casos y fortalecer la auditoría.

Reflexión individual · Oscar Díaz Ulloa

Personalmente, el proyecto me permitió profundizar en LangGraph, la integración de herramientas y el diseño de agentes con memoria y planificación. Mi contribución se enfocó en implementar y validar el flujo del agente, comprendiendo la importancia de combinar IA con lógica determinista para lograr soluciones confiables y trazables.

7. Declaración de uso de inteligencia artificial

Se utilizó **Claude (Anthropic, 2026)** como asistente para apoyar la programación de los módulos del agente, generar los diagramas (Mermaid) y redactar las secciones descriptivas de este informe y del README. El autor revisó, ejecutó y validó el código con las pruebas y los escenarios del repositorio. Las secciones 5 y 6 fueron redactadas por el autor sin apoyo de IA.

8. Referencias

Anthropic. (2024). Building effective agents. https://www.anthropic.com/research/building-effective-agents

Anthropic. (2026). Claude [Modelo de lenguaje de gran escala]. https://claude.ai

Biblioteca del Congreso Nacional de Chile. (2004). Ley 19.966: Establece un régimen de garantías en salud. https://www.bcn.cl/leychile/navegar?idNorma=229834

Biblioteca del Congreso Nacional de Chile. (2024). Ley 21.719: Regula la protección y el tratamiento de los datos personales. https://www.bcn.cl/leychile/navegar?idNorma=1208650

Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. Proceedings of SIGIR '09, 758–759. https://doi.org/10.1145/1571941.1572114

LangChain. (2025). LangGraph: Persistence [Documentación]. https://langchain-ai.github.io/langgraph/concepts/persistence/

LangChain. (2025). LangGraph (Versión 1.x) [Software]. https://github.com/langchain-ai/langgraph

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., … Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459–9474.

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. arXiv. https://arxiv.org/abs/2310.08560

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. ICLR 2023. https://arxiv.org/abs/2210.03629